

Analysis Report

Like

Share

876 people like this. [Sign Up](#) to see what your friends like.

➦ Share this Report

GC log file: **gc-2017_08_19-16_18.log.0**

Duration: **58 min 1 sec 640 ms**

Long Pause 🔥

Our analysis tells that your application is suffering from long running GC events. **8 GC events took more than 10 seconds.** Long running GCs are unfavourable for application's performance.

Read our recommendations to **[reduce long GC pause \(https://blog.gceasy.io/2016/11/22/reduce-long-gc-pauses/\)](https://blog.gceasy.io/2016/11/22/reduce-long-gc-pauses/)**

⌚ Application waiting for resources ⚠️

It looks like your application is waiting due to lack of compute resources (either CPU or I/O cycles). Serious production applications shouldn't be stranded because of compute resources. In **35 GC event(s)**, 'real' time took more than 'usr' + 'sys' time.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2017-08-19T16:30:01	0.33	0.01	1.34
2017-08-19T16:32:42	0.5	0.02	0.85
2017-08-19T16:33:33	0.47	0.01	0.73
2017-08-19T16:34:07	0.61	0.23	11.11
2017-08-19T16:34:37	0.42	0.05	4.22
2017-08-19T16:34:55	0.44	0.04	4.44
2017-08-19T16:35:06	0.91	0.04	5.11
2017-08-19T16:35:16	0.62	0.02	2.49
2017-08-19T16:35:25	1.02	0.05	1.37
2017-08-19T16:36:30	0.61	0.01	1.34
2017-08-19T16:36:40	0.46	0.11	8.46
2017-08-19T16:36:58	0.5	0.15	13.58
2017-08-19T16:37:41	0.58	0.26	27.85
2017-08-19T16:38:35	0.47	0.03	8.88
2017-08-19T16:38:53	0.4	0.02	3.63
2017-08-19T16:40:27	0.55	0.01	1.97
2017-08-19T16:40:50	0.44	0.03	1.62

2017-08-19T16:41:27	0.53	0.04	2.93
2017-08-19T16:41:40	0.49	0.2	16.54
2017-08-19T16:42:24	0.66	0.38	24.99
2017-08-19T16:43:52	0.75	0.04	10.93
15 more events...			

Read our recommendations to [fix this problem \(https://blog.gceasy.io/2016/12/08/real-time-greater-than-user-and-sys-time/\)](https://blog.gceasy.io/2016/12/08/real-time-greater-than-user-and-sys-time/)

🕒 System Time greater than User Time ⚠️

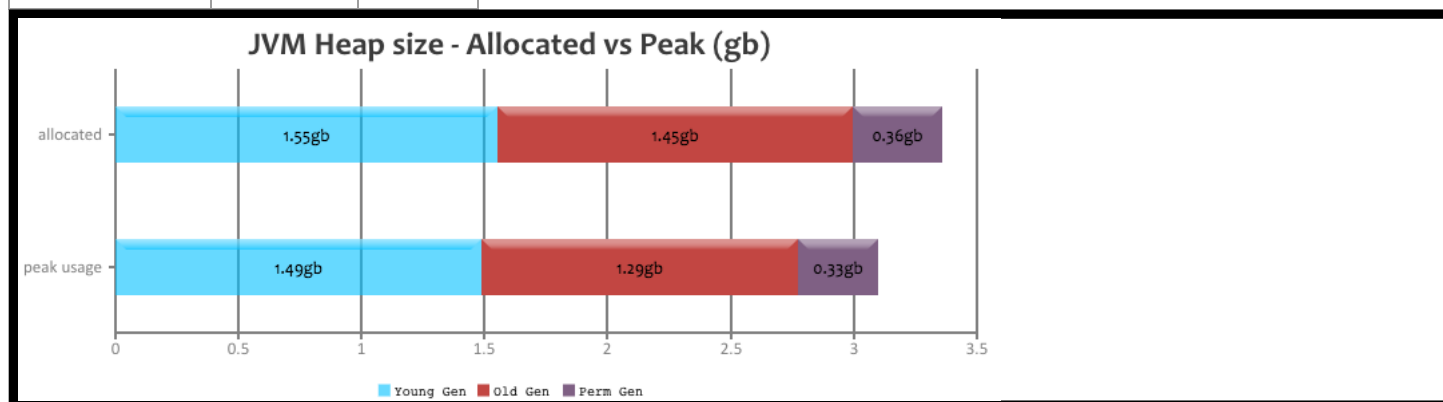
In 1 GC event(s), 'sys' time is greater than 'usr' time. It's not a healthy sign.

Timestamp	User Time (secs)	Sys Time (secs)	Real Time (secs)
2017-08-19T16:27:15	0.26	0.54	0.35

Read our recommendations to [reduce sys time \(https://blog.gceasy.io/2016/12/11/sys-time-greater-than-user-time/\)](https://blog.gceasy.io/2016/12/11/sys-time-greater-than-user-time/)

📊 JVM Heap Size

Generation	Allocated 🕒	Peak 🕒
Young Generation	1.55 gb	1.49 gb
Old Generation	1.45 gb	1.29 gb
Perm Generation	372 mb	335.45 mb
Young + Old + Perm	3.36 gb	2.32 gb



🔍 Key Performance Indicators

(Important section of the report. To learn more about KPIs, [click here \(https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/\)](https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

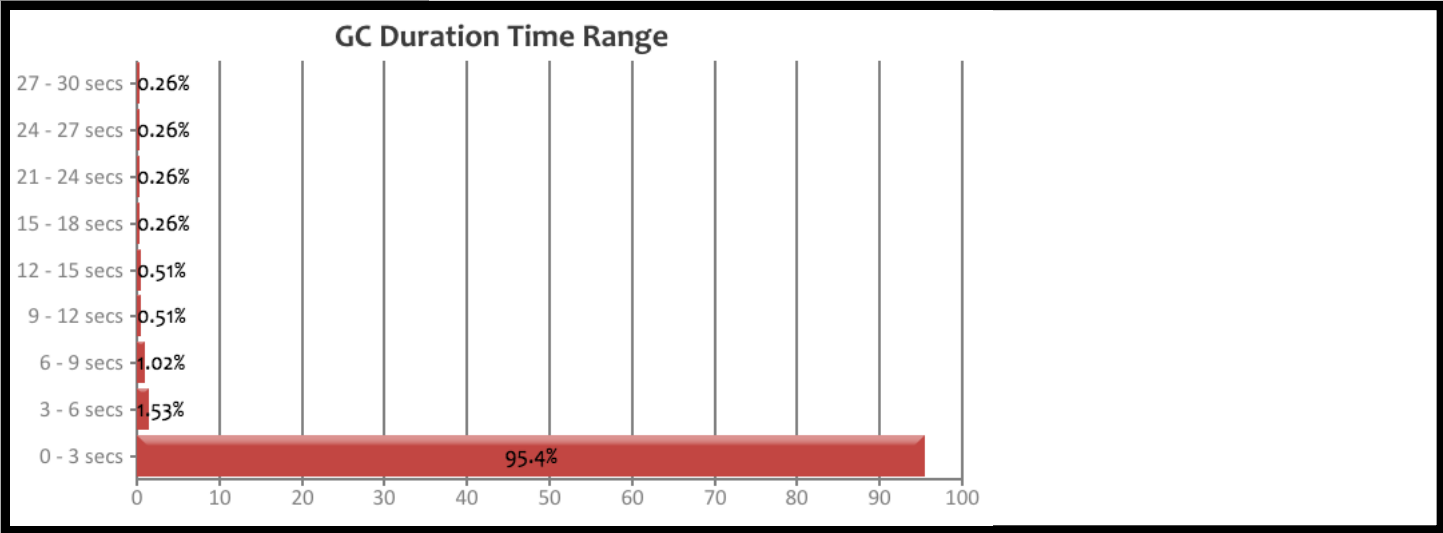
1 Throughput ⓘ : 91.434%

2 Latency:

Avg GC Time ⓘ	757 ms
Max GC Time ⓘ	27 sec 850 ms

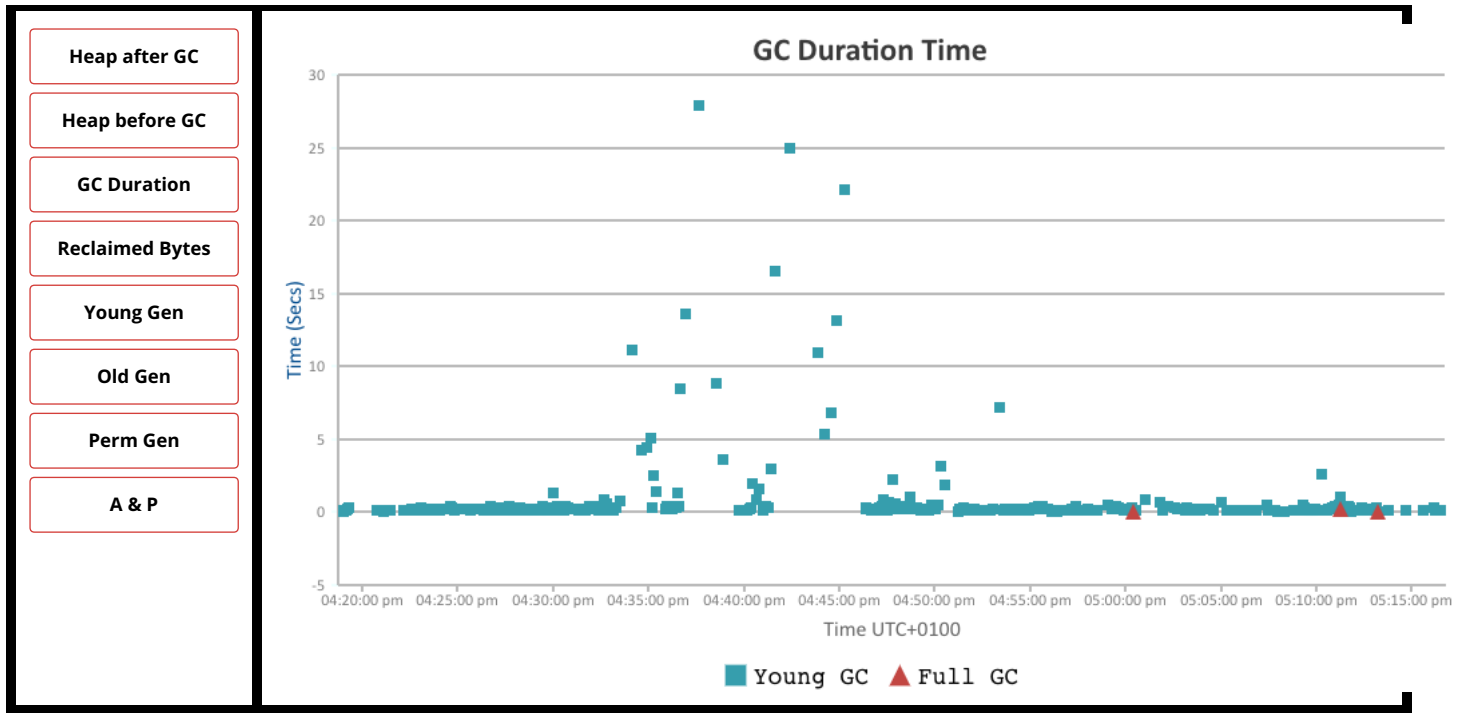
GC Duration Time Range ⓘ:

Duration (secs)	No. of GCs	Percentage
0 - 3	373	95.396%
3 - 6	6	96.931%
6 - 9	4	97.954%
9 - 12	2	98.465%
12 - 15	2	98.977%
15 - 18	1	99.233%
21 - 24	1	99.488%
24 - 27	1	99.744%
27 - 30	1	100.0%

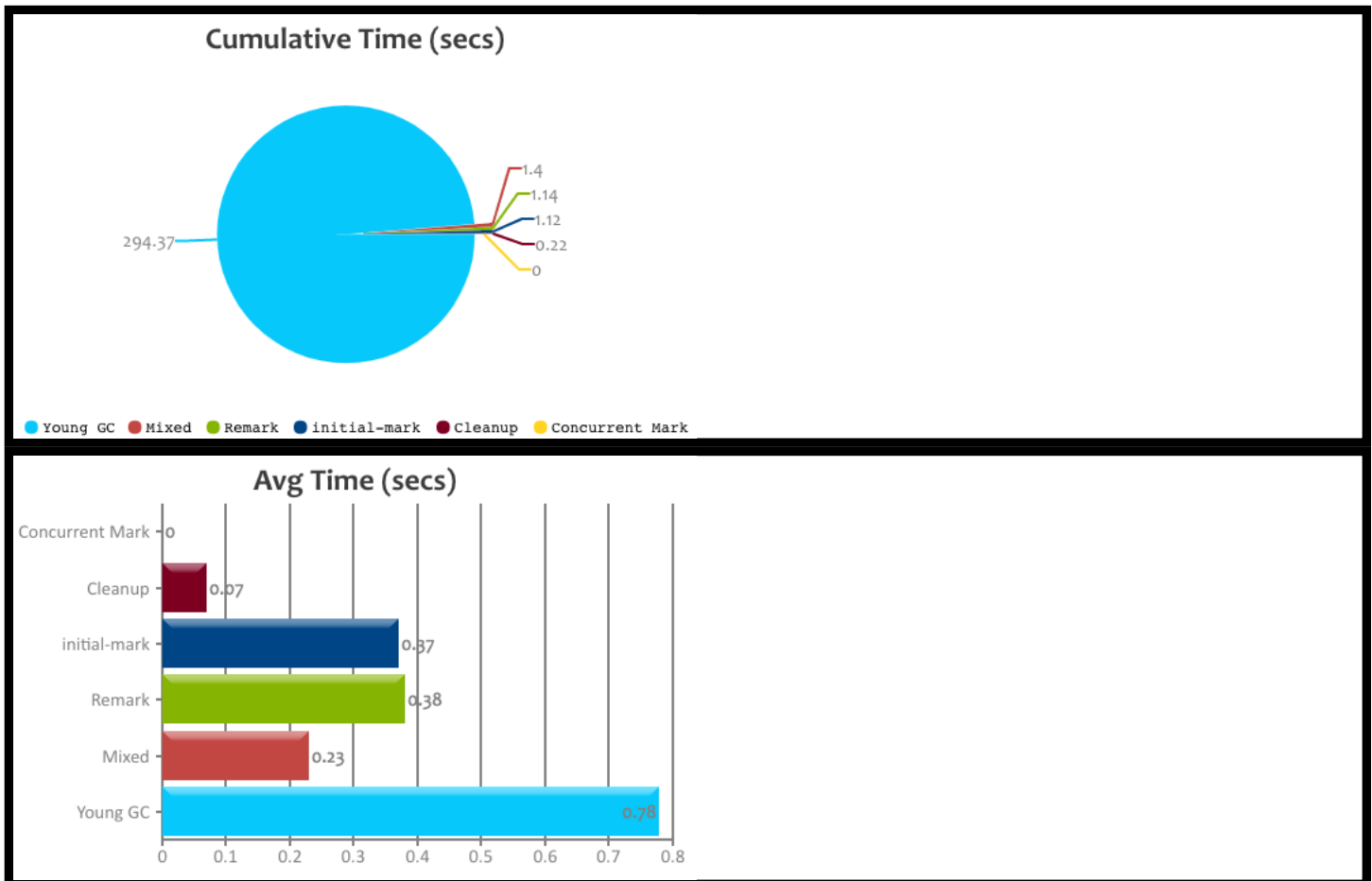


Interactive Graphs

(All graphs are zoomable)



🔍 G1 GC Statistics



	Remark	initial-mark	Mixed	Young GC	Concurrent Mark	Cleanup	Total
Count 📊	3	3	6	376	3	3	394

Total GC Time ⓘ	1 sec 140 ms	1 sec 120 ms	1 sec 400 ms	4 min 54 sec 370 ms	0	220 ms	4 min 58 sec 250 ms
Avg GC Time ⓘ	380 ms	373 ms	233 ms	783 ms	0	73 ms	757 ms
Avg Time std dev	474 ms	118 ms	118 ms	2 sec 808 ms	0	83 ms	2 sec 746 ms
Min/Max Time ⓘ	0 / 1 sec 50 ms	0 / 540 ms	0 / 440 ms	0 / 27 sec 850 ms	0 / 0	0 / 190 ms	0 / 27 sec 850 ms
Avg Interval Time ⓘ	6 min 24 sec 190 ms	6 min 24 sec 633 ms	53 sec 750 ms	9 sec 284 ms	6 min 24 sec 183 ms	6 min 24 sec 182 ms	17 sec 589 ms

⚙ Object Stats

(These are good metrics to include in your performance reports)

Total created bytes ⓘ	220.14 gb
Total promoted bytes ⓘ	4.18 gb
Avg creation rate ⓘ	64.75 mb/sec
Avg promotion rate ⓘ	1.23 mb/sec

💧 Memory Leak ⓘ

No major memory leaks.

(**Note:** there are [8 flavours of OutOfMemoryErrors](https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf) (https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf). With GC Logs you can diagnose only 5 flavours of them(Java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

⏴ Consecutive Full GC ⓘ

None.

🕒 Safe Point Duration ⓘ

(To learn more about SafePoint duration, [click here](https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/) (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

❓ GC Causes ⓘ

(What events caused the GCs, how much time it consumed?)

Not reported in the log.